

JOBSHEET XII

Double Linked List

12.1 Tujuan Praktikum

Setelah melakukan praktikum ini, mahasiswa mampu:

1. Memahami algoritma double linked list;
2. Membuat dan mendeklarasikan struktur algoritma double linked list;
3. Menerapkan operasi penambahan, pencarian, dan penghapusan data pada Double Linked List.

12.2 Percobaan 1: Operasi Penambahan pada Double Linked List

12.2.1 Langkah-Langkah Percobaan

Pada percobaan 1 ini akan dibuat class Mahasiswa, Node, dan DoubleLinkedList. Class tersebut digunakan untuk mengimplementasikan struktur data Double Linked List beserta beberapa operasi dasar seperti:

- penambahan data di awal linked list,
- penambahan data di akhir linked list,
- pencarian data,
- penyisipan node setelah data tertentu,
- dan menampilkan isi linked list.

Langkah-langkah percobaan untuk menerapkan beberapa operasi dasar pada Double Linked List adalah sebagai berikut:

1. Perhatikan diagram class **Mahasiswa<noPresensi>**, **Node<noPresensi>** dan class **DoubleLinkedList<noPresensi>** berikut. Diagram class berikut digunakan sebagai acuan dalam pembuatan program Double Linked List. Perhatikan hubungan antar class dan atribut yang digunakan pada setiap node.

Mahasiswa<noPresensi>
nim: String nama: String kelas: String ipk: Double
Mahasiswa(String nim, String nama, String kelas, Double IPK) tampil()

Node<noPresensi>
data: Mahasiswa prev: Node next: Node
Node(prev:null, data: Mahasiswa data, next: null)

DoubleLinkedList<noPresensi>
head: Node tail : Node
DoubleLinkedLists() isEmpty(): boolean addFirst (): void addLast(): void add(data: Mahasiswa, index: int): void print(): void removeFirst(): void removeLast(): void search(): null insertAfter: void

2. Pada Project yang sudah dibuat pada Minggu sebelumnya, buat folder atau package baru bernama **Jobsheet12** di dalam repository **Praktikum ASD**.

Catatan: Perhatikan setiap perubahan pointer prev dan next pada setiap operasi linked list. Kesalahan perubahan pointer dapat menyebabkan node terputus atau linked list tidak dapat ditelusuri dengan benar.

3. Buat class di dalam paket tersebut dengan nama **Mahasiswa<noPresensi>**. Di dalam class tersebut, deklarasikan atribut sesuai dengan diagram class di atas. Tambahkan konstruktor untuk menginisialisasi seluruh atribut serta method tampil() untuk menampilkan data mahasiswa.

```
public class Mahasiswa {
    String nim;
    String nama;
    String kelas;
    double ipk;

    public Mahasiswa(String nim, String nama, String kelas, double ipk) {
        this.nim = nim;
        this.nama = nama;
        this.kelas = kelas;
        this.ipk = ipk;
    }

    public void tampil() {
        System.out.println(
            "NIM    : " + nim +
            "\nNama  : " + nama +
            "\nKelas : " + kelas +
            "\nIPK   : " + ipk
        );
    }
}
```



4. Buat class di dalam paket tersebut dengan nama **Node<noPresensi>**. Di dalam class tersebut, deklarasikan atribut data, prev, dan next sesuai diagram class. Selanjutnya tambahkan konstruktor sesuai class diagram.

```
public class Node {
    Mahasiswa data;
    Node prev;
    Node next;

    public Node(Mahasiswa data) {
        this.data = data;
        this.prev = null;
        this.next = null;
    }
}
```

5. Buatlah sebuah class baru bernama **DoubleLinkedList<noPresensi>** pada package yang sama dengan **Node**. Pada class **DoubleLinkedList** tersebut, deklarasikan atribut head dan tail pada class DoubleLinkedList.

```
public class DoubleLinkedList {
    Node head;
    Node tail;
```

6. Selanjutnya, buat konstruktor pada class **DoubleLinkedList**.

```
public DoubleLinkedList() {
    head = null;
    tail = null;
}
```

7. Buat method **isEmpty()**. Method isEmpty() digunakan untuk mengecek apakah linked list masih kosong atau sudah memiliki node. Method ini akan bernilai true jika head masih bernilai null.

```
public boolean isEmpty() {
    return head == null;
}
```

8. Kemudian, buat method **addFirst()**. Method addFirst() digunakan untuk menambahkan node baru pada bagian awal linked list. Perhatikan perubahan pointer head dan prev setelah node baru ditambahkan.

```
public void addFirst(Mahasiswa data) {
    Node newNode = new Node(data);
    if (isEmpty()) {
        head = tail = newNode;
    } else {
        newNode.next = head;
        head.prev = newNode;
        head = newNode;
    }
}
```

9. Selain itu pembuatan method **addLast()** digunakan untuk menambahkan node baru pada bagian akhir linked list. Perhatikan perubahan pointer tail dan next setelah proses penambahan dilakukan.



```
public void addLast(Mahasiswa data) {
    Node newNode = new Node(data);
    if (isEmpty()) {
        head = tail = newNode;
    } else {
        tail.next = newNode;
        newNode.prev = tail;
        tail = newNode;
    }
}
```

10. Method **insertAfter()** digunakan untuk menyisipkan node baru setelah node yang memiliki NIM tertentu. Perhatikan perubahan pointer next dan prev pada proses penyisipan node. Urutan perubahan pointer harus benar agar hubungan antar node tidak terputus.

```
public void insertAfter(String keyNim, Mahasiswa data) {
    Node current = head;
    while (current != null && !current.data.nim.equals(keyNim)) {
        current = current.next;
    }
    if (current == null) {
        System.out.println("Data dengan NIM " + keyNim + " tidak ditemukan.");
        return;
    }

    Node newNode = new Node(data);

    // jika current adalah tail, node baru ditambahkan di akhir
    if (current == tail) {
        newNode.prev = current;
        current.next = newNode;
        tail = newNode;
    } else { //node baru disisipkan di tengah
        newNode.prev = current;
        newNode.next = current.next;
        current.next.prev = newNode;
        current.next = newNode;
    }
    System.out.println("Data berhasil disisipkan setelah NIM " + keyNim);
}
```

11. Untuk mencetak isi dari linked lists dibuat method **print()**. Method print() digunakan untuk melakukan traversal dari node head hingga tail dan menampilkan seluruh isi linked list.



```
public void print() {
    if (isEmpty()) {
        System.out.println(x: "Linked List masih kosong.");
        return;
    }

    Node current = head;
    while (current != null) {
        current.data.tampil();
        current = current.next;
    }
}
```

12. Selanjutnya buat class **DoubleLinkedListMain**<noPresensi> untuk mengeksekusi semua method yang ada pada class **DoubleLinkedList**.

```
public static void main(String[] args) {
    Scanner scan = new Scanner(System.in);
    DoubleLinkedList list = new DoubleLinkedList();
    int pilihan;
```

13. Buatlah menu pilihan pada class **DoubleLinkedListMain**. Gunakan struktur perulangan do-while agar program terus berjalan sampai pengguna memilih menu keluar.

```
do {
    System.out.println(x: "\n===== MENU DOUBLE LINKED LIST =====");
    System.out.println(x: "1. Tambah data di awal");
    System.out.println(x: "2. Tambah data di akhir");
    System.out.println(x: "3. Sisipkan data di tengah (setelah NIM)");
    System.out.println(x: "4. Hapus data di awal");
    System.out.println(x: "5. Hapus data di akhir");
    System.out.println(x: "6. Tampilkan data");
    System.out.println(x: "0. Keluar");
    System.out.print(s: "Pilih menu : ");
    pilihan = scan.nextInt();
    scan.nextLine();
```

14. Tambahkan switch-case untuk menjalankan menu pilihan program.

```
switch (pilihan) {
    case 1:
        Mahasiswa mhsAwal = inputMahasiswa(scan);
        list.addFirst(mhsAwal);
        break;
    case 2:
        Mahasiswa mhsAkhir = inputMahasiswa(scan);
        list.addLast(mhsAkhir);
        break;
    case 3:
        System.out.print(s: "Masukkan NIM yang dicari : ");
        String keyNim = scan.nextLine();
        System.out.println(x: "Masukkan data baru: ");
        Mahasiswa dataBaru = inputMahasiswa(scan);
        list.insertAfter(keyNim, dataBaru);
        break;
```



```

        case 4:
            list.removeFirst();
            break;
        case 5:
            list.removeLast();
            break;
        case 6:
            list.print();
            break;
        case 0:
            System.out.println(x: "Program selesai.");
            break;
        default:
            System.out.println(x: "Menu tidak valid.");
    }
} while (pilihan != 0);
scan.close();

```

12.2.2 Verifikasi Hasil Percobaan

Pastikan program berhasil dijalankan dan menghasilkan output yang sesuai seperti pada gambar berikut. Coba lakukan beberapa kombinasi penambahan data untuk memastikan linked list berjalan dengan benar.

```

===== MENU DOUBLE LINKED LIST =====
1. Tambah data di awal
2. Tambah data di akhir
3. Sisipkan data di tengah (setelah NIM)
4. Hapus data di awal
5. Hapus data di akhir
6. Tampilkan data
0. Keluar
Pilih menu : 2
Masukkan NIM   : 123005
Masukkan Nama  : Harry
Masukkan Kelas : 1A
Masukkan IPK   : 3.76

===== MENU DOUBLE LINKED LIST =====
1. Tambah data di awal
2. Tambah data di akhir
3. Sisipkan data di tengah (setelah NIM)
4. Hapus data di awal
5. Hapus data di akhir
6. Tampilkan data
0. Keluar
Pilih menu : 3
Masukkan NIM yang dicari : 123005
Masukkan data baru:
Masukkan NIM   : 123010
Masukkan Nama  : Potter
Masukkan Kelas : 1B
Masukkan IPK   : 3.55
Data berhasil disisipkan setelah NIM 123005

```



```
===== MENU DOUBLE LINKED LIST =====
1. Tambah data di awal
2. Tambah data di akhir
3. Sisipkan data di tengah (setelah NIM)
4. Hapus data di awal
5. Hapus data di akhir
6. Tampilkan data
0. Keluar
Pilih menu : 6
NIM   : 123005
Nama  : Harry
Kelas : 1A
IPK   : 3.76
NIM   : 123010
Nama  : Potter
Kelas : 1B
IPK   : 3.55
```

12.2.3 Pertanyaan

1. Jelaskan perbedaan struktur dan mekanisme traversal antara Single Linked List dan Double Linked List!
2. Perhatikan class **Node**, di dalamnya terdapat atribut next dan prev. Jelaskan fungsi masing-masing atribut tersebut pada proses traversal dan manipulasi node!
3. Perhatikan konstruktor pada class **DoubleLinkedList**. Jelaskan fungsi konstruktor tersebut terhadap kondisi awal linked list!
4. Perhatikan potongan kode berikut:

```
if (isEmpty()) {
    head = tail = newNode;
}
```

Mengapa head dan tail harus menunjuk node yang sama ketika linked list masih kosong?

5. Modifikasi method **print()** agar menampilkan pesan "Linked List masih kosong" ketika tidak terdapat data pada linked list!
6. Modifikasi kode program dengan menambahkan method **printReverse()** untuk menampilkan seluruh data pada Double Linked List secara terbalik, dimulai dari node tail menuju head!

12.3 Percobaan 2: Operasi Penghapusan pada Double Linked List

12.3.1 Langkah-Langkah Percobaan

Pada percobaan 2 akan dibuat beberapa method untuk menghapus node pada Double Linked List. Penghapusan dilakukan melalui:

- node paling depan (removeFirst)
- node paling belakang (removeLast)

Langkah-langkah percobaan untuk menerapkan operasi penghapusan pada Double Linked List adalah sebagai berikut:

1. Buatlah method **removeFirst()** di dalam class **DoubleLinkedList** untuk menghapus node pada bagian awal linked list. Perhatikan perubahan pointer head setelah node pertama dihapus.

```
public void removeFirst() {
    if (isEmpty()) {
        System.out.println(x: "Linked List kosong.");
        return;
    }

    if (head == tail) {
        head = tail = null;
    } else {
        head = head.next;
        head.prev = null;
    }
}
```

2. Tambahkan method **removeLast()** untuk menghapus node pada bagian akhir linked list. Perhatikan perubahan pointer tail setelah proses penghapusan dilakukan.

```
public void removeLast() {
    if (isEmpty()) {
        System.out.println(x: "Linked List kosong.");
        return;
    }

    if (head == tail) {
        head = tail = null;
    } else {
        tail = tail.prev;
        tail.next = null;
    }
}
```

12.3.2 Verifikasi Hasil Percobaan

Pastikan program berhasil dijalankan dan menghasilkan output yang sesuai seperti pada gambar berikut. Coba lakukan beberapa kombinasi penghapusan data untuk memastikan linked list berjalan dengan benar.

```
===== MENU DOUBLE LINKED LIST =====
1. Tambah data di awal
2. Tambah data di akhir
3. Sisipkan data di tengah (setelah NIM)
4. Hapus data di awal
5. Hapus data di akhir
6. Tampilkan data
0. Keluar
Pilih menu : 5
Data berhasil dihapus.
NIM   : 123010
Nama  : Potter
Kelas : 1B
IPK   : 3.55
```




```
===== MENU DOUBLE LINKED LIST =====
1. Tambah data di awal
2. Tambah data di akhir
3. Sisipkan data di tengah (setelah NIM)
4. Hapus data di awal
5. Hapus data di akhir
6. Tampilkan data
0. Keluar
Pilih menu : 6
NIM   : 123005
Nama  : Harry
Kelas : 1A
IPK   : 3.76
```

12.3.3 Pertanyaan Percobaan

1. Perhatikan potongan kode berikut pada method `removeFirst()`:

```
head = head.next;
head.prev = null;
```

Jelaskan fungsi masing-masing statement tersebut pada proses penghapusan node!

2. Modifikasi method `removeFirst()` dan `removeLast()` agar program menampilkan data yang berhasil dihapus!

--- *** ---